

5.1 Tujuan Praktikum

Setelah melakukan praktikum ini diharapkan mahasiswa mampu:

- Mahasiswa mampu membuat algoritma searching bubble sort, selection sort dan insertion sort
- Mahasiswa mampu menerapkan algoritma searching bubble sort, selection sort dan insertion sort pada program

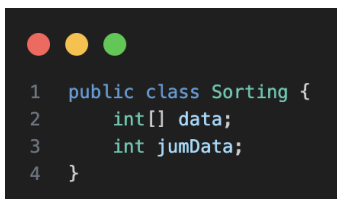
5.2 Praktikum 1 - Mengimplementasikan Sorting menggunakan object

Waktu : 60 menit

5.2.1 Langkah Praktikum 1

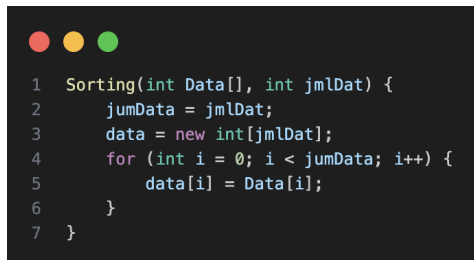
a. SORTING – BUBBLE SORT

- Buat folder baru dengan nama Praktikum05.
- Buat class Sorting<No Presensi>, kemudian tambahkan atribut sebagai berikut:



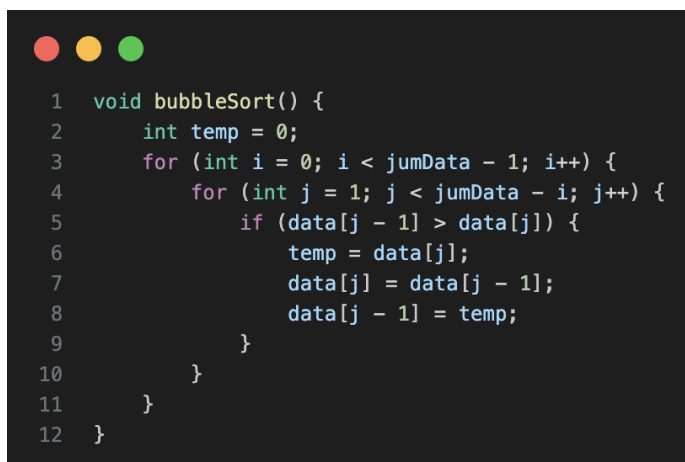
```
1 public class Sorting {
2     int[] data;
3     int jumData;
4 }
```

- Buatlah konstruktor dengan parameter Data[] dan jmlDat



```
1 Sorting(int Data[], int jmlDat) {
2     jumData = jmlDat;
3     data = new int[jmlDat];
4     for (int i = 0; i < jumData; i++) {
5         data[i] = Data[i];
6     }
7 }
```

- Buatlah method bubbleSort bertipe void dan deklarasikan isinya menggunakan algoritma Bubble Sort.



```
1 void bubbleSort() {
2     int temp = 0;
3     for (int i = 0; i < jumData - 1; i++) {
4         for (int j = 1; j < jumData - i; j++) {
5             if (data[j - 1] > data[j]) {
6                 temp = data[j];
7                 data[j] = data[j - 1];
8                 data[j - 1] = temp;
9             }
10        }
11    }
12 }
```

5. Buatlah method tampil bertipe void dan deklarasikan isi method tersebut.

```
1 void tampil() {  
2     for (int i = 0; i < jumData; i++) {  
3         System.out.print(data[i] + " ");  
4     }  
5     System.out.println();  
6 }
```

6. Buat class SortingMain<No Presensi> kemudian deklarasikan array dengan nama a[] kemudian isi array tersebut

```
1 public class SortingMain22 {  
2  
3     public static void main(String[] args) {  
4  
5         int a[] = {20, 10, 2, 7, 12};  
6  
7     }  
8 }
```

7. Buatlah objek baru dengan nama dataurut1 yang merupakan instansiasi dari class Sorting, kemudian isi parameternya

```
1 Sorting22 dataurut1 = new Sorting22(a, a.length);
```

8. Lakukan pemanggilan method bubbleSort dan tampil

```
1 System.out.println("Data awal 1");  
2 dataurut1.tampil();  
3 dataurut1.bubbleSort();  
4 System.out.println("Data sudah diurutkan dengan BUBBLE SORT (ASC)");  
5 dataurut1.tampil();
```

9. Jalankan program, dan amati hasilnya!

5.2.2 Verifikasi Hasil Percobaan

```
Data awal 1  
20 10 2 7 12  
Data sudah diurutkan dengan BUBBLE SORT (ASC)  
2 7 10 12 20  
rizdann@MacBook-Air-rafif AlgoritmaDanStukturData %
```

b. SORTING – SELECTION SORT

1. Pada class Sorting<No Presensi> yang sudah dibuat di praktikum sebelumnya tambahkan method SelectionSort yang mengimplementasikan pengurutan menggunakan algoritma selection sort.

```
1 void selectionSort() {
2     for (int i = 0; i < jumData - 1; i++) {
3         int min = i;
4         for (int j = i + 1; j < jumData; j++) {
5             if (data[j] < data[min]) {
6                 min = j;
7             }
8         }
9         int temp = data[i];
10        data[i] = data[min];
11        data[min] = temp;
12    }
13 }
```

2. Deklarasikan array dengan nama b[] pada kelas SortingMain<No Presensi> kemudian isi array tersebut

```
1 int b[] = {30, 20, 2, 8, 14};
2
```

3. Buatlah objek baru dengan nama dataurut2 yang merupakan instansiasi dari class Sorting, kemudian isi parameternya

```
1 Sorting22 dataurut2 = new Sorting22(b, b.length);
```

4. Lakukan pemanggilan method SelectionSort dan tampil

```
1 System.out.println("Data awal 2");
2 dataurut2.tampil();
3 dataurut2.selectionSort();
4 System.out.println("Data sudah diurutkan dengan SELECTION SORT (ASC)");
5 dataurut2.tampil();
```

5.2.3 Verifikasi Hasil Percobaan

```
Data awal 2
30 20 2 8 14
Data sudah diurutkan dengan SELECTION SORT (ASC)
2 8 14 20 30
rizdann@MacBook-Air-rafif AlgoritmaDanStukturData %
```

c. SORTING – INSERTION SORT

1. Pada class Sorting<No Presensi> yang sudah dibuat di praktikum sebelumnya tambahkan method insertionSort yang mengimplementasikan pengurutan menggunakan algoritma insertion sort.

```
1 void selectionSort22() {
2     for (int i = 0; i < jumData - 1; i++) {
3         int min = i;
4         for (int j = i + 1; j < jumData; j++) {
5             if (data[j] < data[min]) {
6                 min = j;
7             }
8         }
9         int temp = data[i];
10        data[i] = data[min];
11        data[min] = temp;
12    }
13 }
```

2. Deklarasikan array dengan nama c[] pada kelas SortingMain<No Presensi> kemudian isi array tersebut

```
1 int c[] = {40, 10, 4, 9, 3};
```

3. Buatlah objek baru dengan nama dataurut3 yang merupakan instansiasi dari class Sorting, kemudian isi parameternya

```
1 Sorting22 dataurut3 = new Sorting22(c, c.length);
```

4. Lakukan pemanggilan method insertionSort dan tampil

```
1 System.out.println("Data awal 3");
2 dataurut3.tampil();
3 dataurut3.insertionSort22();
4 System.out.println("Data sudah diurutkan dengan INSERTION SORT (ASC)");
5 dataurut3.tampil();
```

5. Jalankan program dan amati hasilnya!

5.2.4 Verifikasi Hasil Percobaan

```
Data awal 3
40 10 4 9 3
Data sudah diurutkan dengan INSERTION SORT (ASC)
3 4 9 10 40
rizdann@MacBook-Air-rafif AlgoritmaDanStukturData %
```

5.2.5 Pertanyaan!

1. Jelaskan fungsi kode program berikut

```
if (data[j-1]>data[j]){  
    temp=data[j];  
    data[j]=data[j-1];  
    data[j-1]=temp;  
}
```

2. Tunjukkan kode program yang merupakan algoritma pencarian nilai minimum pada selection sort!
3. Pada Insertion sort , jelaskan maksud dari kondisi pada perulangan

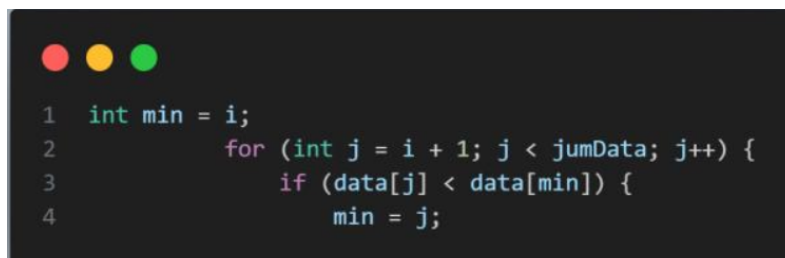
```
while (j>=0 && data[j]>temp)
```

4. Pada Insertion sort, apakah tujuan dari perintah

5.2.5 Jawaban!

1. Kode Tersebut berfungsi untuk Membandingkan dua elemen bersebelahan, Jika elemen kiri lebih besar dari kanan maka akan ditukar, Digunakan dalam **Bubble Sort** untuk mengurutkan data secara ascending

- 2.



```
1  int min = i;  
2      for (int j = i + 1; j < jumData; j++) {  
3          if (data[j] < data[min]) {  
4              min = j;
```

3. Selama index masih valid ($j \geq 0$) dan data lebih besar dari temp maka data digeser ke kanan
4. Menempatkan nilai pada posisi yang benar setelah pergeseran selesai

5.3 Praktikum 2- (Sorting Menggunakan Array of Object)

Waktu : 45 menit

5.3.1 Langkah Praktikum 2 - Mengurutkan Data Mahasiswa Berdasarkan IPK (Bubble Sort)

Perhatikan diagram class Mahasiswa di bawah ini! Diagram class ini yang selanjutnya akan dibuat sebagai acuan dalam membuat kode program class Mahasiswa.

5.3.2 Langkah-langkah Praktikum 2

1. Buatlah class dengan nama Mahasiswa<No Presensi>.
2. Untuk lebih jelasnya class tersebut dapat dilihat pada potongan kode di bawah ini

```
1 public class Mahasiswa {
2
3     String nim;
4     String nama;
5     String kelas;
6     double ipk;
7
8     Mahasiswa() {
9     }
10
11     Mahasiswa(String nm, String name, String kls, double ip) {
12         nim = nm;
13         nama = name;
14         ipk = ip;
15         kelas = kls;
16     }
17
18     void tampilInformasi() {
19         System.out.println("Nama: " + nama);
20         System.out.println("NIM: " + nim);
21         System.out.println("IPK: " + ipk);
22         System.out.println("Kelas: " + kelas);
23     }
24 }
```

3. Buat class MahasiswaBerprestasi<No Presensi> seperti di bawah ini!

```
1 public class MahasiswaBerprestasi {
2
3     Mahasiswa[] listMhs = new Mahasiswa[5];
4     int idx;
5
6 }
```

4. Tambahkan method tambah() di dalam class tersebut! Method tambah() digunakan untuk menambahkan objek dari class Mahasiswa ke dalam atribut listMhs.

```
1 void tambah(Mahasiswa22 m) {
2     if (idx < listMhs.length) {
3         listMhs[idx] = m;
4         idx++;
5     } else {
6         System.out.println("data sudah penuh");
7     }
8 }
```

5. Tambahkan method tampil() di dalam class tersebut! Method tampil() digunakan untuk menampilkan semua data mahasiswa-mahasiswa yang ada di dalam class tersebut!

Perhatikan penggunaan sintaks for yang agak berbeda dengan for yang telah dipelajari sebelumnya, meskipun secara konsep sebenarnya mirip.

```
1 void tampil() {
2     for (Mahasiswa22 m : listMhs) {
3         m.tampilInformasi();
4         System.out.println("-----");
5     }
6 }
```

6. Tambahkan method bubbleSort() di dalam class tersebut!

```
1 void bubbleSort() {
2     for (int i = 0; i < listMhs.length - 1; i++) {
3         for (int j = 1; j < listMhs.length - i; j++) {
4             if (listMhs[j].ipk > listMhs[j - 1].ipk) {
5                 Mahasiswa22 tmp = listMhs[j];
6                 listMhs[j] = listMhs[j - 1];
7                 listMhs[j - 1] = tmp;
8             }
9         }
10    }
11 }
```

7. Buat class MahasiswaDemo<No Presensi>, kemudian buatlah sebuah objek MahasiswaBerprestasi dan buatlah 5 objek mahasiswa kemudian tambahkan semua objek mahasiswa tersebut dengan memanggil fungsi tambah pada objek MahasiswaBerprestasi. Silakan dipanggil fungsi tampil() untuk melihat semua data yang telah dimasukkan, urutkan data tersebut dengan memanggil fungsi bubbleSort() dan yang terakhir panggil fungsi tampil kembali.

```
1 public class MahasiswaDemo22 {
2
3     public static void main(String[] args) {
4
5         MahasiswaBerprestasi22 list = new MahasiswaBerprestasi22();
6         Mahasiswa22 m1 = new Mahasiswa22("123", "Zidan", "2A", 3.2);
7         Mahasiswa22 m2 = new Mahasiswa22("124", "Ayu", "2A", 3.5);
8         Mahasiswa22 m3 = new Mahasiswa22("125", "Sofi", "2A", 3.1);
9         Mahasiswa22 m4 = new Mahasiswa22("126", "Sita", "2A", 3.9);
10        Mahasiswa22 m5 = new Mahasiswa22("127", "Miki", "2A", 3.7);
11
12        list.tambah(m1);
13        list.tambah(m2);
14        list.tambah(m3);
15        list.tambah(m4);
16        list.tambah(m5);
17
18        System.out.println("Data mahasiswa sebelum sorting: ");
19        list.tampil();
20
21        System.out.println("Data Mahasiswa setelah sorting berdasarkan IPK (DESC) : ");
22        list.bubbleSort();
23        list.tampil();
24    }
25 }
```

5.3.3 Verifikasi Hasil Percobaan

Cocokan hasilnya dengan yang terdapat pada tampilan di bawah ini

```
Data mahasiswa Sebelum sorting:
Nama : Zidan
NIM : 123
IPK : 3.2
Kelas : 2A
-----
Nama : Ayu
NIM : 124
IPK : 3.5
Kelas : 2A
-----
Nama : Sofi
NIM : 125
IPK : 3.1
Kelas : 2A
-----
Nama : Sita
NIM : 126
IPK : 3.9
Kelas : 2A
-----
Nama : Miki
NIM : 127
IPK : 3.7
Kelas : 2A
-----
Data mahasiswa setelah sorting berdasar IPK (DESC):
Nama : Sita
NIM : 126
IPK : 3.9
Kelas : 2A
-----
Nama : Miki
NIM : 127
IPK : 3.7
Kelas : 2A
-----
Nama : Ayu
NIM : 124
IPK : 3.5
Kelas : 2A
-----
Nama : Zidan
NIM : 123
IPK : 3.2
Kelas : 2A
-----
Nama : Sofi
NIM : 125
IPK : 3.1
Kelas : 2A
-----
```


5.3.4 Pertanyaan

1. Perhatikan perulangan di dalam bubbleSort() di bawah ini:
 - a. Mengapa syarat dari perulangan i adalah $i < \text{listMhs.length} - 1$?
 - b. Mengapa syarat dari perulangan j adalah $j < \text{listMhs.length} - i$?
 - c. Jika banyak data di dalam listMhs adalah 50, maka berapakah perulangan i akan berlangsung? Dan ada berapa Tahap bubble sort yang ditempuh?
2. Modifikasi program diatas dimana data mahasiswa bersifat dinamis (input dari keyboard) yang terdiri dari nim, nama, kelas, dan ipk!

5.3.4 Jawaban

- a. Karena setiap iterasi akan menempatkan satu elemen terbesar di posisi akhir, jadi tidak perlu sampai elemen terakhir.
- b. Karena bagian belakang array sudah terurut, jadi tidak perlu dibandingkan lagi.
- c. Perulangan i = 49 kali dan tahap bubble sort = 49 kali

2.

```
Masukkan data mahasiswa ke-1
NIM : 12345
Nama : Faqih
Kelas: 2A
IPK : 3.8
Masukkan data mahasiswa ke-2
NIM : 67890
Nama : Raden
Kelas: 2D
IPK : 3.2
Masukkan data mahasiswa ke-3
NIM : 45678
Nama : Ridho
Kelas: 2A
IPK : 3.9
Masukkan data mahasiswa ke-4
NIM : 23456
Nama : Emil
Kelas: 2C
IPK : 3.5

Data Mahasiswa sebelum sorting :
NIM : 12345
Nama : Faqih
Kelas: 2A
IPK : 3.8
-----
NIM : 67890
Nama : Raden
Kelas: 2D
IPK : 3.2
-----
NIM : 45678
Nama : Ridho
Kelas: 2A
IPK : 3.9
-----
NIM : 23456
Nama : Emil
Kelas: 2C
IPK : 3.5
-----
```

5.3.5 Mengurutkan Data Mahasiswa Berdasarkan IPK (Selection Sort)

Waktu : 30 menit

Jika pada praktikum yang sebelumnya kita telah mengurutkan data mahasiswa berdasarkan IPK menggunakan Bubble Sort secara descending, pada kali ini kita akan mencoba untuk menambahkan fungsi pengurutan menggunakan Selection Sort.

5.3.1. Langkah-langkah Percobaan.

1. Lihat kembali class MahasiswaBerprestasi, dan tambahkan method selectionSort() di dalamnya! Method ini juga akan melakukan proses sorting secara ascending, tetapi menggunakan pendekatan selection sort.

```
1 void selectionSort() {
2     for (int i = 0; i < listMhs.length - 1; i++) {
3         int idxMin = i;
4         for (int j = i + 1; j < listMhs.length; j++) {
5             if (listMhs[j].ipk < listMhs[idxMin].ipk) {
6                 idxMin = j;
7             }
8         }
9         Mahasiswa22 tmp = listMhs[idxMin];
10        listMhs[idxMin] = listMhs[i];
11        listMhs[i] = tmp;
12    }
13 }
```

2. Setelah itu, buka kembali class MahasiswaDemo, dan di dalam method main() tambahkan baris program untuk memanggil method selectionSort() tersebut, kemudian panggil method tampil() untuk menampilkan data yang sudah diurutkan!

```
1 System.out.println("Data yang sudah terurut menggunakan SELECTION SORT (ASC)");
2 list.selectionSort();
3 list.tampil();
4 }
```

3. Coba jalankan kembali class MahasiswaDemo, dan amati hasilnya! Apakah kini data mahasiswa telah tampil urut menaik berdasar ipk?

5.3.6 Verifikasi Hasil Percobaan

Pastikan output yang ditampilkan sudah benar seperti di bawah ini

```
Data yang sudah terurut menggunakan SELECTION SORT (ASC)
Nama: Sofi
NIM: 125
Kelas: 2A
IPK: 3.1
-----
Nama: Zidan
NIM: 123
Kelas: 2A
IPK: 3.2
-----
Nama: Ayu
NIM: 124
Kelas: 2A
IPK: 3.5
-----
Nama: Miki
NIM: 127
Kelas: 2A
IPK: 3.7
-----
Nama: Sita
NIM: 126
Kelas: 2A
IPK: 3.9
-----
rizdann@MacBook-Air-rafif AlgoritmaDanStukturData %
```

5.3.7 Pertanyaan

Di dalam method selection sort, terdapat baris program seperti di bawah ini:

```
int idxMin=i;
for (int j=i+1; j<listMhs.length; j++){
    if (listMhs[j].ipk<listMhs[idxMin].ipk){
        idxMin=j;
    }
}
```

Untuk apakah proses tersebut, jelaskan!

5.3.7 Jawaban

kode ini adalah bagian dari selection sort untuk mencari IPK terkecil dalam array mahasiswa

5.4 Mengurutkan Data Mahasiswa Berdasarkan IPK Menggunakan Insertion Sort

Waktu : 30 menit

Yang terakhir akan diimplementasikan Teknik sorting menggunakan Insertion Sort, dengan mengurutkan IPK mahasiswa secara ascending.

5.4.1 Langkah-langkah Percobaan

1. Lihat kembali class MahasiswaBerprestasi, dan tambahkan method insertionSort() di dalamnya. Method ini juga akan melakukan proses sorting secara ascending, tetapi menggunakan pendekatan Insertion Sort.

```
1 void insertionSort() {
2     for (int i = 1; i < ListMhs.length; i++) {
3         Mahasiswa22 Temp = ListMhs[i];
4         int j = i;
5         while (j > 0 && ListMhs[j - 1].IPK > Temp.IPK) {
6             ListMhs[j] = ListMhs[j - 1];
7             j--;
8         }
9         ListMhs[j] = Temp;
10    }
11 }
12
```

2. Setelah itu, buka kembali class MahasiswaDemo, dan di dalam method main() tambahkan baris program untuk memanggil method insertionSort() dan tampil () tersebut!

```
1 System.out.println("Data yang sudah terurut menggunakan SELECTION SORT (ASC)");
2 list.selectionshort();
3 list.tampil();
```

3. Coba jalankan kembali class MahasiswaDemo, dan amati hasilnya! Apakah kini data mahasiswa telah tampil urut menaik berdasar ipk?

5.4.2 Verifikasi Hasil Percobaan

Pastikan output yang ditampilkan sudah benar seperti di bawah ini

```
Nama: ddd
NIM: 222
Kelas: 2c
IPK: 3.4
-----
Nama: aaa
NIM: 123
Kelas: 2c
IPK: 3.6
-----
Nama: zz
NIM: zz
Kelas: 2c
IPK: 3.7
-----
Nama: ii
NIM: 243
Kelas: 2c
IPK: 3.9
-----
Data yang sudah terurut menggunakan INSERTION SORT (ASC)
Nama: rrr
NIM: 123
Kelas: 2c
IPK: 2.9
-----
Nama: ddd
NIM: 222
Kelas: 2c
IPK: 3.4
-----
Nama: aaa
NIM: 123
Kelas: 2c
IPK: 3.6
-----
Nama: zz
NIM: zz
Kelas: 2c
IPK: 3.7
-----
Nama: ii
NIM: 243
Kelas: 2c
IPK: 3.9
-----
```

```
rizdann@MacBook-Air-rafif AlgoritmaDanStukturData %
```

5.5 Latihan Praktikum

Waktu : 45 Menit

```
=== MENU ===
1. Tambah Data
2. Tampil Data
3. Sorting ASC (Bubble Sort)
4. Sorting DSC (Selection Sort)
5. Keluar
Pilihan : 1
Kode   : 234
Nama   : rr
JK (true=Laki-laki, false=Perempuan) : true
Usia   : 23
-----

=== MENU ===
1. Tambah Data
2. Tampil Data
3. Sorting ASC (Bubble Sort)
4. Sorting DSC (Selection Sort)
5. Keluar
Pilihan : 1
Kode   : 1321
Nama   : ii
JK (true=Laki-laki, false=Perempuan) : false
Usia   : 30
-----

=== MENU ===
1. Tambah Data
2. Tampil Data
3. Sorting ASC (Bubble Sort)
4. Sorting DSC (Selection Sort)
5. Keluar
Pilihan : 2
Kode   : 234
Nama   : rr
JK     : Laki-laki
Usia   : 23
-----

Kode   : 1321
Nama   : ii
JK     : Perempuan
Usia   : 30
-----
```

```
=====
=== MENU ===
1. Tambah Data
2. Tampil Data
3. Sorting ASC (Bubble Sort)
4. Sorting DSC (Selection Sort)
5. Keluar
Pilihan : 3
Data berhasil diurutkan ASC!
Kode   : 234
Nama   : rr
JK     : Laki-laki
Usia   : 23
=====
```

```
Kode   : 1321
Nama   : ii
JK     : Perempuan
Usia   : 30
=====
```

```
=== MENU ===
1. Tambah Data
2. Tampil Data
3. Sorting ASC (Bubble Sort)
4. Sorting DSC (Selection Sort)
5. Keluar
Pilihan : 4
Data berhasil diurutkan DSC!
Kode   : 1321
Nama   : ii
JK     : Perempuan
Usia   : 30
=====
```

```
Kode   : 234
Nama   : rr
JK     : Laki-laki
Usia   : 23
=====
```

```
=== MENU ===
1. Tambah Data
2. Tampil Data
3. Sorting ASC (Bubble Sort)
4. Sorting DSC (Selection Sort)
5. Keluar
Pilihan : 5
Keluar...
```

```
❖ rizdann@MacBook-Air-rafif AlgoritmaDanStukturData %
```